
Lawyers & Coders: Technical Documentation

*Aya Abdelrahman, Davide Hanna, Khushi Jain
Melody Scales, Yujia Liu*

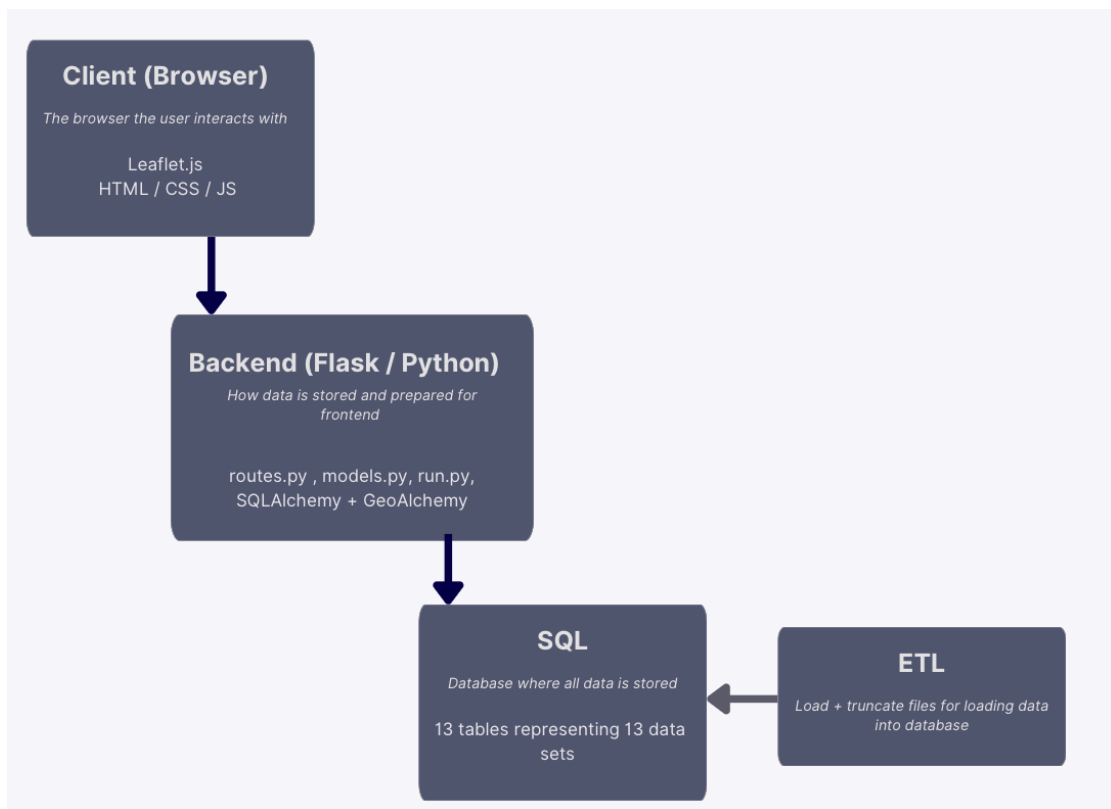
1. System Overview

To view the code, please refer to our Github:

https://github.com/Yuchiaaa/Interactive_Map_Website

Architecture:

The system follows a four-tier architecture:



Tier 1 - Front-end

[Leaflet.js](#) renders the interactive map in the browser (HTML / CSS/ JS)

This is the layer users interact with, when navigating the map, turning layers on/off, using the filtering features, and making exports.

Tier 2 - Backend

Flask/Python handles data processing and serves it to the frontend via [routes.py](#), [models.py](#), [run.py](#). It acts as the bridge between the database and front-end.

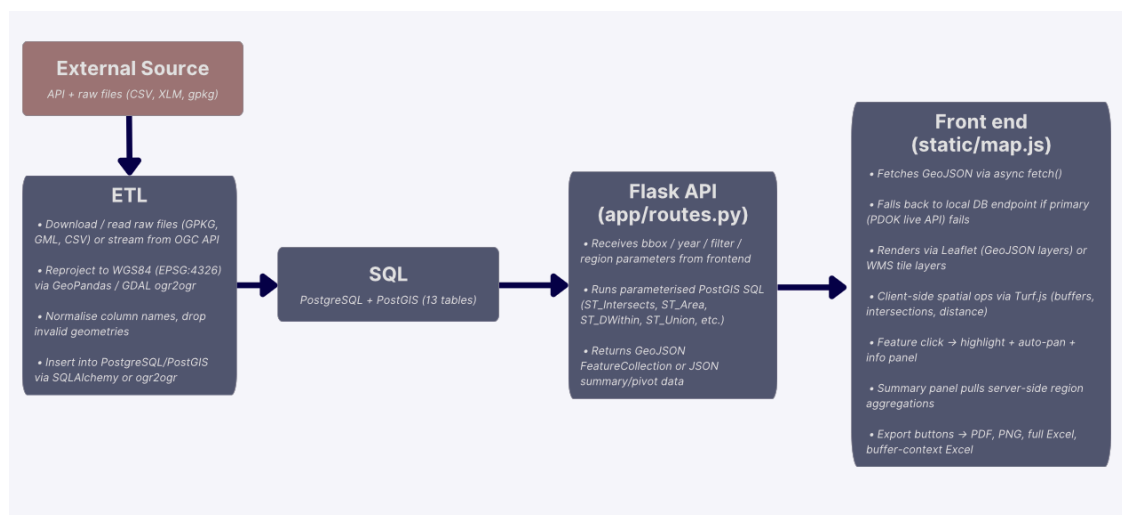
Tier 3 - SQL

PostgreSQL with the PostGIS extension stores all spatial data across 13 tables, each representing one dataset. SQLAlchemy and GeoAlchemy2 manage the connection between the Python backend and the database, enabling spatial queries to run efficiently.

Tier 4 - ETL pipeline

The ETL (Extract, Transform, Load) pipeline is responsible for sourcing, cleaning, and loading data into the database. It downloads / streams raw files from external sources, standardises formats and geometries, and inserts the processed data into the tables, keeping the database up to date as new sources are introduced

Data flow



Raw data enters the system from external sources, including APIs and raw files in formats like CSV, GML, and GPKG. The ETL pipeline downloads or streams this data, reprojects it to WGS84, normalises column names, and removes invalid geometries before loading everything into a PostgreSQL/PostGIS database.

The Flask API ([routes.py](#)) sits between the database and the frontend. It receives spatial parameters from the user, such as when clicking filters for a year or zooming in on a region, and runs PostGIS queries against the database, returning GeoJSON feature collections or JSON summary data.

The frontend ([map.js](#)) fetches this GeoJSON and renders it via Leaflet as vector layers or WMS tiles. If the primary PDOK live API is unavailable, it falls back to the local database. Any client-side spatial analysis, such as buffers, intersections, distance calculations, run via turf.js without hitting the server. Users can click features to highlight them and pull up an info panel, while the summary table aggregates information within a zoom-level view. Exports are available as PDF, PNG, and Excel in both full and buffer-context formats.

Deployment note: the app now ships with a Dockerfile, .dockerignore, and Procfile running Gunicorn (gunicorn run:app --workers 2 --bind 0.0.0.0:\$PORT), and has been prepared for production hosting on Digital Ocean. The earlier "development server only, not suitable for hosting yet" limitation no longer applies.

2. Technology Stack

Component Overview

Below is a table summarising the tools and libraries used for the mapping prototype, with the associated layer, version, and purpose listed. These were chosen based on compatibility with leaflet, data structure, desired outcomes and features, as well as technical expertise.

Layer	Technology	Version	Purpose
Frontend map	Leaflet.js	1.9.4	Map rendering, layer management, tile display
Frontend spatial	Turf.js	6.x	Client-side buffer, intersection, distance calculations
Frontend charts	Chart.js	4.4.0	Analytics dashboard charts, reading from /api/dashboard_stats and /api/brp_trend
Frontend export	jsPDF	2.5.1	PDF generation from map canvas
Frontend export	html2canvas	1.4.1	Map screenshot capture (feeds both PDF and PNG export)
Frontend export	SheetJS (xlsx)	0.18.5	Client-side XLSX export (pivot tables, buffer context)
Frontend UI	JS / HTML5 / CSS3	-	Layer panel, info panel, filter controls, highlight system
Backend	Flask	3.x	HTTP routing, API endpoints, template rendering
Backend ORM	SQLAlchemy + Flask-SQLAlchemy	2.x	Database connection management
Backend spatial ORM	GeoAlchemy2	0.17.1	PostGIS geometry column type mapping
Backend data	Pandas	-	Excel export data processing, dashboard aggregation
Backend export	openpyxl	-	Server-side XLSX generation with metadata
Production server	Gunicorn	23.0.0	WSGI server for deployment
Containerization	Docker	-	Dockerfile + .dockerignore for PaaS deployment (app container only; PostGIS runs separately)
Database	PostgreSQL + PostGIS	14+ / 3.x	spatial data storage, indexed geometry queries

ETL spatial	GeoPandas	-	Coordinate reprojection, geometry validation
ETL spatial	GDAL / ogr2ogr	-	Bulk loading of GPKG/GML files into PostGIS
ETL utility	pyogrio	-	Fast reading of GeoPackage layer metadata
Fonts	Google Fonts	-	UI typography

Data sources (PDOK, BAG, etc.)

Below is a table summarising the datasets used in the mapping prototype, their sources, and how each is accessed and processed through the data pipeline. Where API is used, a local database fallback is implemented to ensure the tool remains functional if the external service is unavailable.

Dataset	Provider	Access Method
BRP Crop Parcels	RVO / PDOK	GeoPackage file download (annual)
BAG Buildings	Kadaster / PDOK	GeoPackage file download
Natura 2000	Ministerie van LNV / PDOK	API + DB fallback
Nature Network NL	Provincies / Nationaal Georegister	ZIP download
Kadastrale Kaart	Kadaster / Nationaal Georegister	API + WMS tiles
Bestuurlijke Grenzen	Kadaster / PDOK	GeoPackage file download
KRD Livestock Farms & Stallen	Kernregistratie Dierverblijven	Manual CSV export
Pesticides Atlas	Bestrijdingsmiddelen Atlas	CSV download (annual)
Health Facilities	HOTOSM / OpenStreetMap	GeoPackage download
Schools	DUO	CSV with geocoded coordinates
Water Hydrography	Waterschappen / PDOK	Streamed via paginated OGC API
WFD Surface Water	Rijkswaterstaat / PDOK	GML file download
Waterschappen	Unie van Waterschappen / PDOK	OGC API

3. Project Structure

Folder structure

Below shows an overview of the folder structure. This is especially important for replicating the environment, viewing the codebase, and committing changes to the map.

Interactive_Map_Website/

- **[run.py](#)** (Application entry point)
- **Dockerfile** (Production container build (Gunicorn, GDAL, port 8000))
- **.dockerignore**
- **Procfile** (Gunicorn config for PaaS deployment)
- **Requirements.txt**

- **app/**
 - **__init__.py** (Flask app, DB init,)
 - **models.py** (SQLAlchemy)
 - **routes.py** (All API endpoints and Flask routes)
 - **utils.py** (Utility helpers)

- **etl/**
 - **master_sync.py** (coordinates automated ETL loaders in sequence)
 - **load_brp.py** (BRP crop parcels ETL)
 - **load_bag.py** (BAG buildings ETL)
 - **load_natura2000.py** (Natura 2000 areas ETL)
 - **load_nnn.py** (NNN areas ETL)
 - **load_kadastralekaart.py** (Cadastral parcels ETL)
 - **load_grenzen.py** (Administrative boundaries ETL)
 - **load_krd.py** (KRD livestock farms ETL)
 - **load_stallen.py** (KRD individual stallen ETL)
 - **load_pesticides.py** (Pesticides measurements ETL)
 - **load_healthcare.py** (Health facilities ETL)
 - **load_schools.py** (Schools ETL)
 - **load_hydrography.py** (Hydrography ETL)
 - **load_waterschappen.py** (Water authority borders ETL)
 - **load_wfd_surface_water.py** (WFD surface water ETL)
 - **truncate_*.py** (Per-dataset table truncators)

- **static/**
 - **css/style.css** (All application styles)
 - **js/map.js** (Frontend)

- **templates/**
 - **home.html** (Home page)
 - **index.html** (Main map interface)

- dashboard.html (Analytics dashboard)

4. Backend Documentation

API Endpoints

Endpoint	Method	Parameters	Returns
/	GET	-	Renders home.html
/index	GET	-	Renders index.html (main map)
/dashboard	GET	-	Renders dashboard.html
/api/brp_parcel	GET	bbox, year, gemeente	GeoJSON FeatureCollection with area_ha
/api/brp_pivot	GET	bbox, year, gemeente	JSON - hectares per crop category
/api/brp_trend	GET	-	JSON - hectares per crop per year
/api/brp_gemeenten	GET	-	JSON list of all gemeente names in BRP
/api/buffer_context	GET	lat, lng, radius_km	JSON - gemeente, province, nearest Natura 2000, N2000 within radius
/api/summary_regions	GET	-	JSON - all gemeenten and provinces available for region filtering
/api/summary/<dataset>	GET	region_type, region_name	JSON - aggregated statistics for a dataset within a region or the full Netherlands
/api/gemeente_boundary	GET	gemeente	GeoJSON Feature - polygon boundary of a single gemeente
/api/kad_gemeenten	GET	-	JSON list of gemeente names for Kadastrale Kaart
/api/bag_buildings	GET	bbox	GeoJSON FeatureCollection
/api/natura2000_areas	GET	bbox, buffer_km	GeoJSON with area + buffer polygons
/api/grenzen	GET	bbox	GeoJSON - municipality/province/national boundaries
/api/krd_farms	GET	bbox	GeoJSON - KRD livestock farms
/api/krd_stallen	GET	bbox	GeoJSON - KRD individual stallen
/api/pesticides	GET	bbox	GeoJSON - pesticide measurement stations
/api/health_facilities	GET	bbox	GeoJSON - health facility points
/api/schools	GET	bbox	GeoJSON - school points

/api/waterschappen	GET	bbox	GeoJSON - water authority boundaries
/api/available_years	GET	-	JSON - available years per temporal dataset
/api/dashboard_stats	GET	-	JSON - aggregated stats for all dashboard charts
/api/nnn	GET	bbox	GeoJSON - Nature Network NL polygons
/api/hydrography	GET	bbox	GeoJSON - combined water hydrography lines
/api/hydrography_main	GET	bbox	GeoJSON - primary water hydrography lines
/api/hydrography_other	GET	bbox	GeoJSON - secondary water hydrography lines
/api/wfd_surface_water	GET	bbox	GeoJSON - WFD surface water bodies
/api/kadastralekaart	GET	bbox	GeoJSON - cadastral parcels
/api/export_excel	POST	active layers, view context	Excel workbook download

Data processing pipeline

Data sources

Thirteen datasets were added into the system, covering crop parcels, cadastral land registration, Natura 2000 protected areas, buildings and addresses, animal housing emissions, schools, healthcare facilities, pesticides, and water quality. These come in a range of file formats including CSV, JSON, GeoJSON, and GeoPackage files.

Data entry

Data enters the system in two ways: either pulled directly from APIs or downloaded as raw files. Both routes feed into the same raw data storage, ensuring all datasets are available in one place before processing begins.

Data pre-processing

The raw data is cleaned, validated, merged, and transformed into a consistent format. This step removes errors, fixes invalid geometries, and standardizes column names so that all datasets can be stored and queried together reliably.

Cleaned database

The processed data is stored in a PostgreSQL/PostGIS database. This is the central glue of the whole system. All spatial queries and analysis run against this database.

Interactive Web Application

This is what users see and interact with. The map visualisation tool allows users to select layers, apply filters, and run spatial calculations such as buffer zones and proximity analysis, all without needing any technical expertise.

Analytics and Reports

Once a user has completed their analysis, results can be exported as PDF reports, PNG map screenshots, or Excel files. These outputs are formatted to be used directly in legal proceedings or shared with stakeholders.

5. Known Technical Limitations

1. *No automated tests*

There are currently no automated tests for the codebase. This means that when changes are made to the data pipeline or API, there is no safety net to catch errors, something that worked before can break without any warning.

2. *KRD coverage is three provinces only*

The KRD dataset covers only Gelderland/Twente, Limburg, and Noord-Brabant.

3. *BRP historical depth may be limited*

The goal is full 2009-2025 coverage, but local storage constraints may mean only recent year(s) are actually loaded at any given time. Check `/api/available_years` for what's currently in the database rather than assuming full coverage.

4. *PDF/PNG export may produce blank WMS tiles*

html2canvas cannot capture cross-origin tile images (Natura 2000, NNN, Kadastrale Kaart WMS) due to browser CORS/taint restrictions.

5. *Viewport-based data loading creates export gaps*

Features just outside the visible viewport are excluded from exports even if they fall within an active buffer zone.

6. *No persistent session state*

Page refresh resets all layers, buffers, filters, and selections.

7. *No authentication or access control*

Every page and API endpoint, including the bulk Excel export, is fully public. No login, no API key, no rate limiting. Anyone with the URL can query or export the entire underlying dataset. Given this tool's output is meant for legal proceedings, this is probably the most important omission to flag.

8. *ETL automation is partial*

`etl/master_sync.py` only orchestrates 7 of 13 datasets (BRP, pesticides, health facilities, WFD surface water, hydrography, NNN, Kadastrale Kaart). BAG, Natura 2000, Grenzen, KRD/stallen, Schools, and Waterschappen still require manually running each `load_*.py` script by hand, with file paths hardcoded/edited in the `if __name__ == "__main__"` block each time

9. ***Silent failure on DB-unavailable startup***

Create_app() in app/__init__.py:23 catches exceptions from db.create_all() and just prints a warning to the console. The app can boot successfully even if the database is unreachable, and will only fail later when a request actually hits the DB.

10. ***Inconsistent duplicate-load protection***

load_bag.py, load_natura2000.py, load_waterschappen.py, and load_brp.py all check for existing data before inserting and skip if found. load_schools.py has no such check; re-running it would duplicate rows in the schools table with no warning.